AFRINEXT · AUTHENTICATION · THE PREVIEW BUG REPORT, AND BREVO

Where The Code Went

A bug report from the preview: signup works, the verification screen appears, no email arrives, and the Render log shows nothing at all. **The missing email was not a bug — no provider was configured and none was claimed.** What made it look like one was four real defects, and this note is mostly about those. Then, with them fixed, Brevo — read out of its own SDK rather than remembered, and behind an interface that had to be split first so that connecting it could not take phone sign-in down with it.

Branch **claude/create-app-repository-1cxsih** · Code at **6fda7cc** · Since **b334904**

Unit tests **781 passed, 20 skipped** · Browser tests **57 passed** · Mutations **32 caught, 1 no-op**

Migrations **none** · Real email sent **none**

READ THIS FIRST: WHAT IS PROVED HERE AND WHAT IS NOT

Nothing in this work has ever spoken to Brevo. This environment's egress proxy blocks `api.brevo.com` and `developers.brevo.com` alike — verified, `CONNECT … 403`. So the adapter is written against Brevo's own published SDK and tested against a local HTTP server that checks the request the way that SDK says Brevo checks it.

What that establishes is exact: the adapter forms the request Brevo's contract describes, over real HTTP, and handles the statuses and error body that contract names. What it does not establish is that a real Brevo account accepts it. **Section G lists what only the first real send can prove**, and nothing here is described as more than it is.

A The report, and what it actually was

Reported from `afrinext-preview.onrender.com`: an account is created, the screen says *"Vérifiez votre adresse e-mail"*, nothing reaches the Yahoo inbox, and the Render log between 07:04:51 and 07:05:31 contains only health checks. No send, no `ConsoleSender`, no error.

I reproduced it locally under the preview's exact configuration — production build, `NODE_ENV=production`, `ALLOW_CONSOLE_SENDER=yes` — rather than reasoning about it, and measured this:

```
POST /api/v1/auth/email/verify -> 200  {"sent":true}      ← at SIGNUP
   (no API call at all when /verify-email loads)
POST /api/v1/auth/email/verify -> 429  retryAfterMs 46176  ← the RESEND
```

And the server log for the whole session, in full:

```
{"level":"info","msg":"domain user provisioned",...}
[ConsoleSender] NOT DELIVERED — would send to diag@example.com: … : 495050
```

> **THE ANSWER, IN TWO SENTENCES**
>
> **No email would ever arrive, because no provider was configured** — that was the state the previous note described and it was working as documented. The code *was* written to the Render log, **at signup**, which is not the moment the log was being watched. The person was looking for an email the system was never going to send, at the wrong time, in the one place it actually existed.

That much was not a defect. What made it indistinguishable from one was four things that were.

#	DEFECT	WHY IT MATTERED HERE
D1	Every refusal in the verification route was silent	Six consecutive 429s produced **zero** log lines. An operator watching the log during a resend saw nothing — which is exactly what was reported.
D2	The screen announced a send it had not made	*"Nous avons envoyé un code à X"* on every render, whether or not anything was ever sent.
D3	Five presses of "resend" spent the whole hour	And answered with a 45-minute wait, having issued no code at all.
D4	The signup send was fire-and-forget	A failure there was silent, and the next screen claimed success anyway.

One correction to the brief I was given, worth recording because it would have sent a reviewer to the wrong file: the audit asked about Better Auth's `emailOTP` plugin, its

`sendVerificationOTP` callback and `guardedSend`. Those all exist — and **this screen never calls any of them.** They are Better Auth's email *sign-in* path. Afrinext's verification runs through its own route and its own `MessageSender`.

B D1 · A refusal you cannot see is a broken sender

`issueAndSend` returned `limited(verdict)` with no log and no audit. So did the route's 401, its already-verified exit and its unreachable-address exit. The only path that said anything was the one nobody hits.

Meanwhile `guardedSend` — the phone path, in the same package — has logged *and* audited every refusal since Phase 1. **I wrote two refusal paths and instrumented one.** That is the whole of D1.

What a refusal writes now

```
{"level":"warn","msg":"email code not issued","component":"auth.email",
 "reason":"rate_limited","channel":"email","purpose":"email_verification",
 "address":"d******************@example.com",
 "used":2,"limit":1,"retryAfterMs":8446}
```

Plus an `auth.email.rate_limited` audit row against the Afrinext identity, carrying the same fields. Four facts an operator needs: what refused, how far over, and how long.

> **WHY THE ADDRESS IS MASKED AND THE DOMAIN IS NOT**
>
> A log line is a copy of the data living somewhere with different access rules and a different retention. *"aicha@yahoo.com asked for a code at 07:04"* is a fact about a person that operations does not need in order to do its job.
>
> The domain survives because it is genuinely operational — it answers "is one mail provider being hammered?" The local part keeps its first character and its length, which is enough for the person holding the address to recognise a line as theirs and not enough to reconstruct one that is not. The code is never written at all.

Out of scope but worth naming: **`guardedSend` still logs its identifier in full.** The same privacy defect, on the phone path, which is the channel that matters most at launch. Untouched here, and the next thing to fix.

C D2 & D4 · A screen that says only what it knows

`/verify-email` opened with *"Nous avons envoyé un code à X"* unconditionally. The page issues nothing — measured: **zero API calls on load**. It was a claim about an event on a different screen, which this page had no way to know had happened, and which it would have been flatly wrong about after a refused send.

It now names the address and asks for the code, and reports separately whether one is genuinely outstanding. `hasLiveChallenge()` asks the database: unconsumed, unexpired, right purpose.

> AND IT STILL SENDS NOTHING ON LOAD
>
> Deliberate. An automatic send there would spend one of the five an hour every time somebody re-read the screen, and the cooldown would then refuse the resend they actually meant to make. The button is the only thing that sends, and a browser test asserts the page makes no request when it opens.

D4 — the signup send is awaited

`void requestEmailVerification()`. Blocking nothing and telling nobody are different things: a refused or failed send left somebody on a screen announcing a code, with no code and no way to know why. It is awaited now, and because the verification screen reads real state there is nothing fragile to carry across — a send that did not happen simply leaves no pending code, and the screen says so and offers the button.

Verification still gates nothing. The account is created, the dashboard is reachable, and a delivery failure is not a failure of signup.

D D3 · Five presses, and forty-five minutes

The most expensive of the four, and the one a person actually meets. Measured before the fix, pressing "Renvoyer le code" five times — the obvious response to an email that has not arrived:

PRESS	ANSWER	RETRY-AFTER
1	429	8 s

PRESS	ANSWER	RETRY-AFTER
2	429	6 s
3	429	5 s
4	429	4 s
5	429	**2703 s** — 45 minutes

`consumeAll` stops at the first refusal, and `consume` counts refused requests too — deliberately, so hammering a blocked bucket cannot drain it early. With the hourly per-address cap ordered **first**, every cooldown refusal therefore also spent one of the five sends. Signup (1) plus five presses (5) exhausted the hour **without a single code being issued.**

The order is the fix. No limit was raised.

RULE	WHY IT SITS THERE
`per IP, hourly`	The flood gate, and now it counts **every** attempt including ones the later rules refuse. **Strictly stronger than before**: previously an address-level refusal returned before the IP bucket was ever touched, so flooding one address from one connection cost an attacker nothing per-IP.
`cooldown, 1/window`	Refuses a too-early resend *without reaching* the hourly rule.
`per address, hourly`	Reached only by a request that will actually issue a code. The five are five codes, not five button presses.

And the wait now reaches the person

The server always sent `retryAfterMs` in the body and `Retry-After` in the header. The client threw both away and showed *"Trop de demandes. Réessayez plus tard."* — which tells somebody to guess, and guessing means pressing the button again.

Measured after the fix, same five presses:

```
press 1 → 200 {"sent":true}
press 2 → 429  Retry-After: 9   retryAfterMs: 8446
presses 3–5 → the button is disabled; "Réessayez dans 7, 6, 5 secondes."

rate_limit_counters:
  email:cooldown:email_verification:addr:…  |  2
  email:send:email_verification:addr:…      |  1   ← one credit, one code
```

The countdown runs off a wall-clock deadline rather than a decremented counter: a backgrounded mobile tab does not receive its intervals on time, and drift there means

telling somebody to keep waiting after the server has stopped refusing. `aria-live="polite"`, not assertive — a screen reader should hear the wait settle, not be interrupted four times a second.

> **PROVED AT THE REAL LIMITS, IN A BROWSER**
>
> A test does the whole journey with the reviewed production numbers — signup, resend refused, countdown shown, button disabled, **the cooldown waited out**, resend accepted, second code delivered, second credit spent. It takes 39.9 seconds because it actually waits.

E Brevo, read rather than remembered

You validated Brevo. The instruction was to consult only its official current documentation and guess no endpoint, header, payload or behaviour.

Both of Brevo's documentation hosts are blocked from here — `developers.brevo.com` and `api.brevo.com`, verified. So rather than reconstruct the API from memory, which is exactly what was forbidden, I cloned and read **Brevo's own published PHP SDK**: `github.com/getbrevo/brevo-php`, SDK 5.0.2, commit `1b371f9` — code Brevo generates from its own specification.

FACT	FILE IT WAS READ FROM
`https://api.brevo.com/v3`	`src/Environments.php`
`POST smtp/email`	`src/TransactionalEmails/TransactionalEmailsClient.php`
header `api-key` (lower-case, not Authorization)	`src/Brevo.php`
`sender · to · subject · textContent · htmlContent · replyTo · headers`	`…/SendTransacEmailRequest.php`
`messageId · messageIds`	`…/SendTransacEmailResponse.php`
`{ code?, message }`	`src/Types/ErrorModel.php`
retries 408/429/5xx; **no default timeout**	`README.md`

> **THE NEAR-MISS THAT JUSTIFIES READING THE SOURCE**

A web search offered `/v3/smtp/emails` as the send endpoint. **It is not.** That path is the **GET** that lists sent mail; the send is `POST /v3/smtp/email`, singular. One character, and a guess would have shipped it.

What the SDK does not state is not invented

The exact success status is not asserted — the SDK treats 2xx and 3xx alike, and so does the adapter. Neither is the body of a 401, nor the account's rate limits, nor what happens when the sender domain is unverified. None of it is written down anywhere I could read, so none of it is claimed.

Four decisions inside the adapter

- **The timeout is required, not optional.** The SDK configures none and says so. An unbounded call sits inside the signup path holding somebody on a spinner for as long as the socket stays open. Default 10 s, and a test proves it fires against a server that never answers.
- **The 70-character sender name is checked at construction.** The limit is Brevo's. Discovered at send time it means every code silently failing, for every account, until somebody reads a log — so it stops the process starting instead.
- **`Idempotency-Key` carries the challenge id.** `issueChallenge` retires any live code and writes exactly one row, so one id means one code means one message. A retry after a lost response delivers one email rather than two working codes minutes apart.
- **`textContent` and `htmlContent` both.** One line of text either way; the HTML part exists because a multipart message is treated better by spam filters, and a code in a spam folder is indistinguishable from one never sent — which is precisely the failure this whole note is about.

> NOTHING SENSITIVE LEAVES, AND HERE THAT IS UNUSUALLY LOAD-BEARING
>
> The thing being sent **is** a secret: `EmailMessage.body` holds the verification code. So the user-facing message is fixed text; only Brevo's own `code` and `message` are read from a failure, so an unexpected field cannot smuggle anything into a log; the address is masked; and a transport failure is reduced to its error *name* before it is written, because a fetch failure can carry the request and the request carries the code.

F The split that keeps phone sign-in alive

This was the finding that had to be decided before any Brevo code was written, and you decided it.

`MessageSender` carried `sendSms` and `sendEmail`, and **one instance served both** the email flow and — through `createAuth` — the phone OTP flow, which is the launch path in Niger. A single Brevo sender would have had to answer for `sendSms`, and the only honest answer is a throw. Connecting email would have taken phone sign-in down in production.

```
EmailSender  { sendEmail }      ← BrevoSender, ConsoleSender
SmsSender    { sendSms }        ← ConsoleSender only; no provider chosen
MessageSender extends both      ← unchanged for every existing call site
CompositeSender                 ← id "email:brevo/sms:console"
```

The composite's id names both senders, so an audit row now says which one actually carried the message rather than naming a bundle — more than the single id recorded before, and exactly the question asked when somebody reports that a code never arrived.

Verified in one process, in production mode

`NODE_ENV=production`, `EMAIL_PROVIDER=brevo`, a placeholder key:

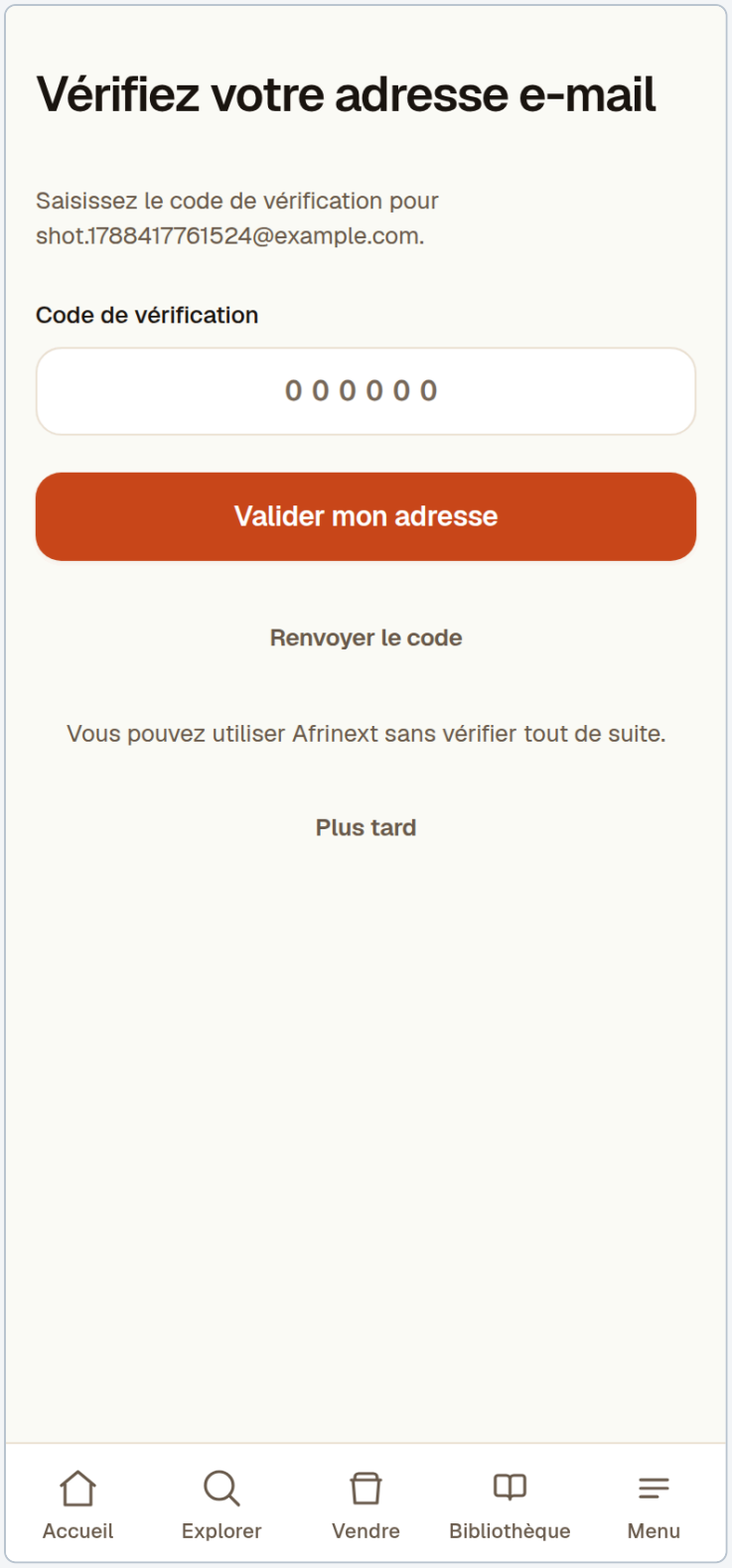
```
{"level":"error","msg":"brevo refused the message","component":"auth.email.brevo",
 "status":403,"brevoCode":null,"brevoMessage":null,"to":"b*********@example.com"}

[ConsoleSender] NOT DELIVERED — would send to +22795835464: Afrinext: 944867
```

The first line is the adapter meeting the egress proxy's 403 and handling a non-JSON error body without inventing one. **The second is the phone channel still delivering, in the same process, with Brevo selected** — the whole point of the split, demonstrated rather than asserted. Grepped over the entire log: the API key appears **0** times, the full address **0** times, and no code at all.

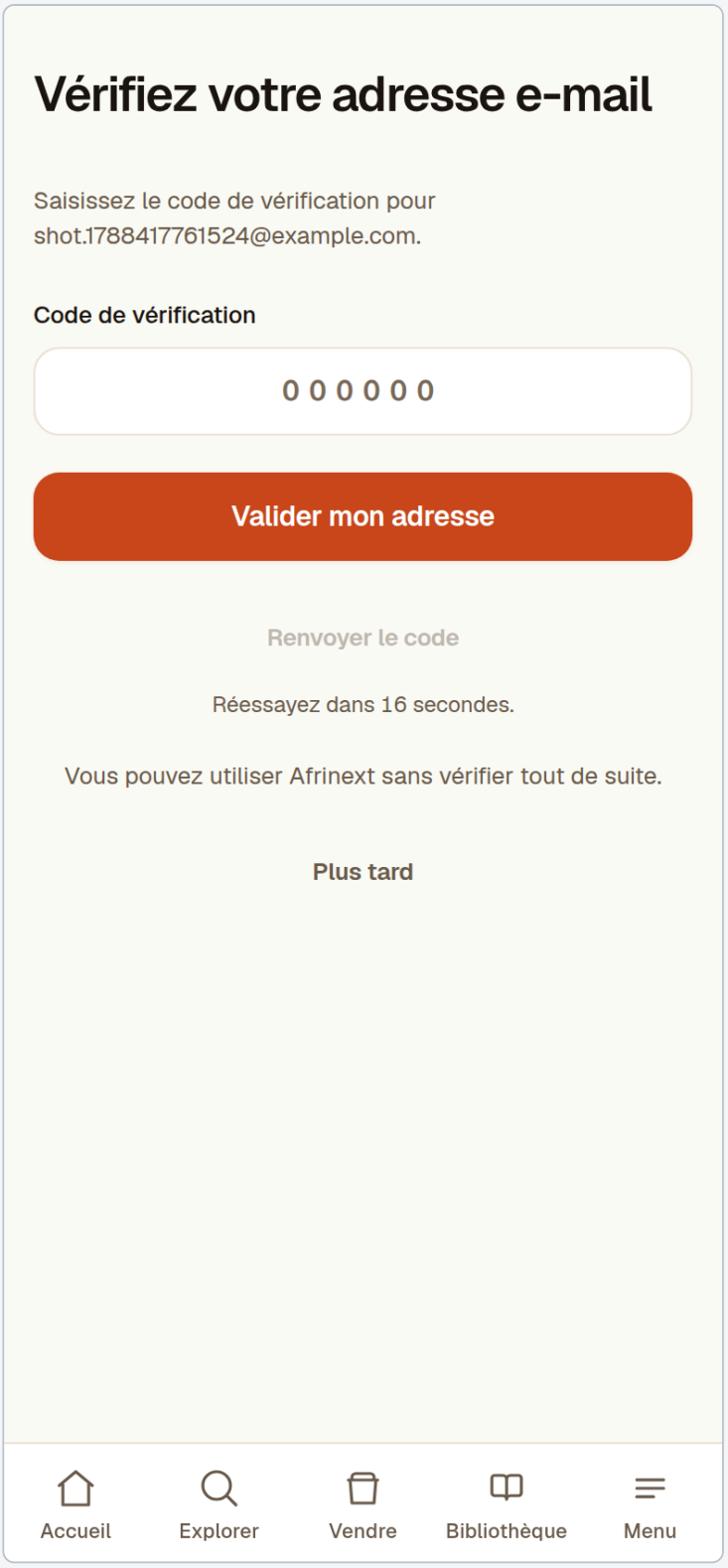
> **AN OPERATIONAL CONSEQUENCE, STATED RATHER THAN LEFT TO BE DISCOVERED**
>
> `ALLOW_CONSOLE_SENDER=yes` is **still required** on Render even with Brevo sending real email, because the SMS half is still the console and `ConsoleSender` refuses production without it. Removing it will stop the service booting. It comes out the day an SMS provider is chosen, and not before. `render.yaml` now says so in those words.

What it looks like

From the running production build at 390 px. The Sahel design is untouched; only the words and the state changed.



The verification screen. It names the address and asks for the code — it does not claim to have sent one, because it did not, and it cannot know whether the screen before it did.



A resend refused. The wait is on screen and the button is disabled, instead of “réessayez plus tard” and an invitation to press again until the hour is gone.

G Tests, mutations, and what only Render can prove

SUITE	RESULT	COVERS
packages/core	781 passed, 20 skipped	41 files. 34 new tests across the four fixes and Brevo.
auth/brevo.test.ts	18	The request, against a real local server; 401/429/5xx; the timeout; the leak assertions.
auth/select-sender.test.ts	13	Every selector branch, the production refusal, the composite, the SMS channel's independence.
auth/email-identity.test.ts	42	Observability, quota accounting, the idempotency key, `hasLiveChallenge`.
pnpm test:e2e	57 passed	Including the real-limits cooldown journey. **Phone sign-in unaffected.**
lint / typecheck / build	clean	`--max-warnings 0`, all six packages.

Mutations

Two matrices, run one mutation at a time in a detached worktree with its own database, each against **the whole core suite**.

MATRIX	DESIGNED	CAUGHT	COVERS
D1–D4	13	13	Silent refusal, unaudited refusal, unmasked address, the old rule order, a dropped cooldown, four ways for `hasLiveChallenge` to lie.
Brevo	20	19 + 1 no-op	Wrong header, wrong path, sender/to swapped, dropped idempotency key, swallowed error, no timeout, unchecked name limit, five distinct leak routes, selector fallback, SMS following the email provider.

Three results worth more than the score

G6 SURVIVED THE FIRST PASS — MY TEST WAS WEAK

Replacing `idempotencyKey: challenge.challengeId` with `undefined` broke nothing. `ConsoleSender` ignores the field, so every test in the file passed whether or not the key was threaded, and the adapter tests only proved the adapter sends a key *it is handed*. A recording sender now compares the key against the row the challenge store actually wrote.

G11 IS A NO-OP, AND IT TOOK ME TWO ATTEMPTS TO SEE IT

Giving the error class an optional `detail` parameter that no call site passes changes nothing observable. That is not a surviving defect — it is a badly designed mutation, twice. **G11b** expresses the same property properly, throwing with the status, Brevo's message and the address, and six tests kill it.

G12 FIRST REPORTED PATTERN NOT FOUND, WHICH WAS MY HARNESS

A run killed by a command timeout had left a mutation applied in the worktree. Against a clean tree it is caught. Worth knowing before trusting a resumed matrix — and worth reporting rather than quietly re-running until the number looked right.

Render — what to set

VARIABLE	VALUE	WHERE IT COMES FROM
`EMAIL_PROVIDER`	`brevo`	already in `render.yaml`
`EMAIL_BREVO_API_KEY`	your key	**`sync: false`** — Render asks; never committed
`EMAIL_FROM`	the sending address	`sync: false`
`EMAIL_FROM_NAME`	`Afrinext`	already in `render.yaml`, ≤ 70 chars
`EMAIL_REPLY_TO`	optional	`sync: false`; unset omits the field entirely
`ALLOW_CONSOLE_SENDER`	`yes`	**still required** — see section F

It is the **API key** from Settings → SMTP & API, **not** the SMTP relay password: two different credentials, and only one works here. **No real credential exists anywhere in the repository** — the only `xkeysib-` strings are the test server's fixed placeholder and the deliberate wrong-key case.

What only the first real send can prove

1. **That a real account accepts this request.**

   Everything comes from the published SDK. A real account is the only judge, and this note claims nothing more.

2. **The success status, the 401 body, and the account's quotas.**

   Not written down anywhere reachable, so not asserted.

3. **Deliverability — SPF, DKIM, DMARC on the sending domain.**

Do this *before* switching over. An unverified sender produces an accepted API call and a code filed as spam — **a symptom identical to the one that opened this note.**

4 Egress from Render to `api.brevo.com`, and the real latency.

The send is awaited inside signup since D4. The timeout is 10 s; adjust once measured.

> **HOW TO READ THE FIRST ATTEMPT**
>
> Sign up once and watch the log. A line reading `brevo refused the message` with a `brevoCode` tells you exactly what Brevo objects to — and it does so without exposing the key, the address or the code. If instead you see nothing at all, that is now a bug rather than a normal Tuesday, which is the point of section B.

Nothing is merged. The work is on `claude/create-app-repository-1cxsih` and waits for your word.

↑ BACK TO THE TOP